

GRADUATION PROJECT DOCUMENTATION

Medical Question-Answering Chatbot Using Retrieval-Augmented Generation (RAG)

Prepared by:

Youssef Mohamed
Kholoud Abdelmohsen
Salma Emad
Mohamed Ramadan
Bishoy Rafat
Bishoy Emad

Digital Egypt Pioneers Initiative (DEPI)

2026

1. Abstract

This document presents the comprehensive technical documentation for a Medical Question-Answering Chatbot developed as a graduation project under the Digital Egypt Pioneers Initiative (DEPI). The system is designed to provide accurate, context-aware answers to medical questions by combining two primary sources of medical knowledge: a structured clinical dataset of diseases and symptoms, and a comprehensive medical reference textbook. The architecture employs a Retrieval-Augmented Generation (RAG) pipeline, which retrieves semantically relevant knowledge from a pre-built vector database and passes it as context to a large language model (LLM) for answer generation. The project encompasses the full lifecycle of an artificial intelligence application, from raw data acquisition and preprocessing, through embedding-based vector indexing, to inference-time retrieval and natural language generation. The system is capable of answering a wide variety of medical queries with citations to the source material, making it both informative and verifiable.

2. Introduction

The rapid proliferation of health-related inquiries on the internet has underscored the need for reliable, AI-powered tools that can provide medically grounded responses. General-purpose search engines and static health websites often fail to address nuanced patient questions or to integrate multiple sources of medical knowledge coherently. In contrast, a well-designed medical chatbot can synthesize information from verified sources and deliver targeted, concise answers.

This project presents such a system. Rather than training a domain-specific model from scratch, which requires enormous computational resources and labeled data, the team adopted a RAG architecture. RAG systems augment the inference capabilities of a pre-trained LLM by dynamically retrieving relevant documents at query time and conditioning the model's response on that retrieved content. This approach ensures that answers are grounded in actual source material rather than solely in the parametric knowledge of the model, thereby significantly reducing the risk of hallucination.

The project integrates two complementary sources of medical information: a structured CSV dataset containing symptom-disease mappings derived from clinical records, and a comprehensive medical textbook in PDF format. Both sources are processed, chunked, embedded into dense vector representations, and stored in a FAISS vector index. At query time, the most semantically similar chunks are retrieved, formatted into a prompt, and passed to the LLaMA 3.1 language model hosted via the Groq inference platform.

2.1 Objectives

- To build a reliable medical question-answering system grounded in verified knowledge sources.
- To implement a full RAG pipeline covering data ingestion, vector storage, retrieval, and language model inference.
- To evaluate the system's retrieval quality using quantitative metrics including Hit Rate and Recall at K.
- To deploy an accessible user interface enabling non-technical users to interact with the system.

2.2 Scope

The system is designed to answer questions about diseases, their symptoms, diagnostic criteria, treatments, and complications. It does not provide personal medical advice, diagnose individual patients, or replace consultation with a licensed medical professional. The scope is deliberately bounded to ensure that the system remains informative and safe.

3. System Architecture

The system follows a modular, pipeline-based architecture consisting of four major phases: data acquisition and preprocessing, embedding and vector database construction, retrieval, and response generation. Each phase is independent but feeds directly into the next, enabling the team to develop and test components in isolation.

3.1 High-Level Pipeline

1. **Data Sources:** A CSV clinical dataset and a medical PDF textbook are loaded as raw inputs.
2. **Document Loading and Preprocessing:** Source files are parsed into structured documents, irrelevant columns and metadata are removed, and the content is prepared for chunking.
3. **Text Chunking:** Documents are divided into overlapping fixed-size text segments to preserve local context while enabling granular retrieval.
4. **Embedding Generation:** Each text chunk is converted into a dense numerical vector using a pre-trained sentence transformer model.
5. **Vector Database Construction:** Vectors are stored in a FAISS index with deduplication to allow fast approximate nearest-neighbor search.
6. **Retrieval:** At query time, the user's question is embedded and used to query the FAISS index, returning the top-k most semantically similar chunks.
7. **Prompt Construction:** Retrieved chunks are formatted into a structured prompt that instructs the LLM to answer the question using only the provided context and to cite sources.
8. **Language Model Inference:** The LLaMA 3.1 8B model, accessed through the Groq API, processes the prompt and generates a concise, cited answer.
9. **User Interface:** A Gradio-based web interface accepts user queries and displays model responses.

4. Data Sources

The selection of appropriate data sources is foundational to any knowledge-grounded AI system. For this project, two distinct but complementary sources were chosen to maximize the breadth and depth of medical knowledge available to the retrieval system.

4.1 Structured Clinical Dataset (CSV)

The primary structured data source is a CSV file containing clinical records that map patient-reported symptoms to confirmed disease diagnoses. This dataset was sourced from Kaggle and preprocessed by the team before being integrated into the pipeline.

The original dataset contained multiple columns, several of which were redundant or uninformative for the purpose of question answering. Specifically, the second, fourth, and fifth columns were dropped using pandas, retaining only the columns most directly relevant to the symptom-disease relationship. This preprocessing step reduced noise and improved the quality of the text representations ultimately stored in the vector database.

Each remaining row in the cleaned dataset was then serialized into a flat text string by concatenating column names and their corresponding values in a key-value format, separated by a pipe character. This transformation converted the structured tabular data into a form amenable to natural language processing, while preserving the semantic relationship between field names and their values. For example, a record might be serialized as: Disease: Acne | Symptom_1: Blackheads | Symptom_2: Pus_filled_pimples. The resulting text strings were wrapped into LangChain Document objects with a metadata field indicating their source as the CSV dataset.

4.2 Medical Reference Textbook (PDF)

The second data source is a comprehensive medical textbook in PDF format, titled `Medical_book.pdf`. This book provides detailed, narrative-style information about a wide range of diseases, their pathophysiology, symptoms, diagnostic criteria, treatment protocols, and

potential complications. It serves as the primary source of contextual and explanatory medical knowledge within the system.

The textbook was downloaded programmatically from Google Drive using the `gdown` library, enabling reproducibility of the data ingestion step within the Kaggle notebook environment. The book was loaded using the `PyPDFLoader` from the LangChain community library, which parsed the PDF page by page and returned each page as a separate LangChain Document object. Each document contained the page text as its content and the source file path and page number as metadata.

A subsequent filtering step was applied to normalize the document metadata. Only the source file path and the page number were retained, and all other metadata fields introduced by the PDF loader were discarded. This standardization ensures consistency across documents from different sources and simplifies the metadata handling during retrieval and citation generation.

5. Data Preprocessing and Text Chunking

Raw documents, whether derived from the CSV dataset or the PDF textbook, cannot be stored directly in a vector database with optimal retrieval performance. Documents that are too long exceed the context length of embedding models and introduce irrelevant information into retrieved chunks. Documents that are too short may lack the necessary context to answer a question. Text chunking addresses this challenge by dividing documents into segments of controlled length.

5.1 Chunking Strategy

The project employs the `RecursiveCharacterTextSplitter` from the LangChain text splitting library. This splitter attempts to divide text at natural linguistic boundaries, using a priority-ordered list of separators including double newlines, single newlines, spaces, and individual characters. It only falls back to less natural separators when the text cannot be split at the preferred level while satisfying the size constraint.

The chunking configuration used in this project specifies a chunk size of 500 characters and a chunk overlap of 50 characters. The chunk size of 500 characters was selected as a balance between providing sufficient context within each chunk and maintaining specificity during retrieval. Chunks that are too short may not contain enough information to answer a question, while chunks that are too long may include irrelevant content that dilutes the similarity signal.

The chunk overlap of 50 characters ensures that information near the boundary between two consecutive chunks is not lost. Without overlap, a sentence that spans the boundary between two chunks would be split, potentially rendering both chunks incomplete. With a 50-character overlap, the tail of each chunk is repeated at the beginning of the next, preserving continuity of meaning across boundaries.

5.2 Document Merging

After chunking both sources independently, the resulting chunk lists are merged into a single unified collection. The PDF-derived chunks and the CSV-derived chunks are concatenated to form a combined list of text segments. This unified collection is what is ultimately embedded and stored in the vector database, enabling the retrieval system to draw on both sources without distinction.

The final combined corpus consists of thousands of chunks, covering a broad spectrum of medical conditions from both the structured symptom-disease perspective of the CSV dataset and the narrative clinical perspective of the medical textbook. Each chunk retains its metadata, including source identifier and, for PDF-derived chunks, the page number from the original book. This metadata is critical for generating citations in the final answer.

6. Text Embedding

Embedding is the process of converting text into dense numerical vectors in a high-dimensional semantic space, such that texts with similar meanings are placed close together in that space.

This representation enables efficient similarity search, which is the foundation of the retrieval component of the RAG pipeline.

6.1 Embedding Model

The embedding model selected for this project is sentence-transformers/all-MiniLM-L6-v2, loaded through the HuggingFaceEmbeddings class provided by the LangChain community library. This model was developed by the Sentence Transformers project and fine-tuned specifically for generating semantically meaningful sentence and paragraph-level embeddings.

The all-MiniLM-L6-v2 model is a compact, distilled version of a larger transformer architecture. It produces 384-dimensional embedding vectors, which provide sufficient representational capacity for semantic similarity tasks while consuming significantly less memory and computational time than larger models. This makes it particularly well-suited for a project environment such as Kaggle, where GPU memory and disk space are constrained.

The model was selected for three primary reasons. First, it has a well-established performance profile on semantic similarity benchmarks, demonstrating strong ability to capture the meaning of diverse text inputs. Second, its small size allows it to run efficiently without a GPU, making it practical for the batch embedding of thousands of chunks during index construction. Third, it is freely available through the Hugging Face model hub, requiring no API credentials or licensing agreements.

6.2 Embedding Process

During vector database construction, every text chunk in the combined corpus is passed through the embedding model to produce a corresponding 384-dimensional vector. These vectors encode the semantic content of the chunk in a way that is independent of exact wording, allowing the retrieval system to find relevant chunks even when the user's query uses different terminology than the stored document.

The same embedding model is used at query time to encode the user's question into a vector representation. The similarity between the query vector and each stored chunk vector is then computed to identify the most relevant chunks. Because both the chunks and the query are embedded by the same model, the similarity computation is meaningful and consistent.

7. Vector Database Construction

A vector database is a specialized data store optimized for storing and searching high-dimensional vectors. Unlike traditional relational databases that match records based on exact values, a vector database retrieves records based on approximate semantic similarity, enabling it to find text passages that are conceptually related to a query even when they share no common keywords.

7.1 FAISS

The vector database used in this project is built on FAISS (Facebook AI Similarity Search), an open-source library developed by Meta AI Research. FAISS is designed for efficient similarity search and clustering of dense vectors. It implements a range of indexing algorithms that trade off between search speed, memory consumption, and recall accuracy. For this project, the default FAISS flat index was used via the LangChain FAISS integration, which performs exact nearest-neighbor search by comparing the query vector against every stored vector. While exact search has linear time complexity, it guarantees perfect recall and is practical for corpora of the size used in this project.

7.2 Deduplication

Before inserting chunks into the FAISS index, a deduplication step is performed to prevent redundant information from inflating the index and biasing retrieval. Each chunk is assigned a unique identifier generated using the UUID version 5 algorithm, which produces deterministic UUIDs based on the content of the chunk combined with its metadata fields including source, page number, and row identifier.

If two chunks produce the same UUID, they are considered duplicates, and only the first occurrence is inserted into the index. This ensures that the index contains a clean, non-redundant representation of the knowledge corpus, which in turn ensures that retrieval results are diverse rather than returning multiple copies of the same content.

7.3 Persistence

Once constructed, the FAISS index is saved to disk using the `save_local` method. The index is serialized to a local directory containing the vector data file and a supporting index configuration file. In the Kaggle environment, this directory was then compressed into a ZIP archive and uploaded as a Kaggle dataset, making it available for use in the inference notebook without needing to rebuild the index from scratch. In the Google Colab inference environment, the index is loaded from Google Drive using the `load_local` method with the `allow_dangerous_deserialization` flag enabled, as required by the LangChain FAISS loader for security compliance.

8. Retrieval Mechanism

The retrieval component is the bridge between the static knowledge stored in the vector database and the dynamic query processing performed by the language model. Its purpose is to identify, from the full corpus, the small subset of text chunks most likely to contain information relevant to the user's question.

8.1 Retriever Configuration

The FAISS vector store is converted into a LangChain retriever object using the `as_retriever` method. The retriever is configured with a search type of similarity, meaning that it retrieves chunks based on cosine similarity between the embedded query vector and the stored chunk vectors. The number of chunks returned per query is set to five ($k=5$), a value chosen to provide sufficient context to the language model without overwhelming the prompt with tangentially related information.

8.2 Query Processing

When a user submits a question, the retriever first encodes the question text into a 384-dimensional vector using the same all-MiniLM-L6-v2 model used during index construction. This ensures that the semantic space of the query is aligned with the semantic space of the stored chunks. The FAISS index then performs an approximate nearest-neighbor search to identify the five chunks whose vectors are most similar to the query vector.

Each retrieved chunk is returned along with its metadata, which includes the source identifier and, for book-derived chunks, the page number. This metadata is used during prompt construction to provide the language model with citation information, which it is instructed to include in its response.

9. Prompt Engineering

Prompt engineering is the practice of designing the input to a language model in a way that elicits the desired output behavior. In a RAG system, the prompt must convey the retrieved context, the user's question, and a set of behavioral instructions that constrain how the model should use the context.

9.1 System Prompt Design

The system prompt used in this project establishes the model's role and behavioral constraints. The model is instructed to act as a medical assistant, to use only the retrieved context when formulating its answer, and to explicitly state that it does not know the answer if the context is insufficient. It is further instructed not to fabricate information under any circumstances, to keep its answer concise, limited to a maximum of three sentences, and to include a citation indicating the source name and page number of the information used.

These constraints serve several important purposes. Restricting the model to the retrieved context prevents it from drawing on its parametric knowledge in ways that might introduce inaccuracies

or hallucinations. The explicit instruction to admit ignorance rather than speculate is a safety mechanism that discourages the generation of plausible-sounding but incorrect medical information. The citation requirement ensures that every answer is traceable back to a specific passage in the source material, enabling users or reviewers to verify the response.

9.2 Context Formatting

Before the prompt is assembled, the retrieved chunks are formatted into a structured context block. Each chunk is prefixed with its source identifier and page number in a standardized format. The chunks are then concatenated with a separator line, creating a multi-source context block that the language model can parse to determine which piece of information came from which source. This formatting is critical for enabling the model to generate accurate citations that reflect the actual provenance of each piece of information in its answer.

10. Language Model and Inference

The final step in the RAG pipeline is the generation of a natural language answer by a large language model. The language model takes the assembled prompt, which includes the formatted retrieved context and the user's question, and generates a response that synthesizes the relevant information into a coherent, concise answer.

10.1 LLaMA 3.1 8B Instant

The language model used for inference is LLaMA 3.1 8B Instant, a large language model developed by Meta AI and made available through the Groq inference API. LLaMA 3.1 is a member of the LLaMA family of open-weight models, which are transformer-based autoregressive language models trained on large multilingual text corpora.

The 8B parameter variant was selected for this project because it offers a strong balance between response quality and inference speed. Larger models, such as the 70B or 405B variants, might produce higher-quality outputs in some scenarios, but their inference latency would be

prohibitive for an interactive chatbot application. The 8B model produces coherent, contextually appropriate responses at low latency, making it suitable for real-time question answering.

10.2 Groq Inference Platform

Groq is a specialized AI inference platform built around the Groq Language Processing Unit (LPU), a custom-designed chip optimized for the sequential, memory-bound computation patterns characteristic of autoregressive language model inference. Unlike general-purpose GPUs, which are designed for massively parallel matrix operations, the Groq LPU is architected to minimize memory latency, enabling it to generate tokens at substantially higher rates.

The use of Groq in this project means that even though the LLaMA 3.1 8B model has eight billion parameters, inference latency is low enough for practical conversational use. The model is accessed through the LangChain Groq integration using the ChatGroq class, which provides a consistent interface compatible with the rest of the LangChain pipeline. Authentication is handled via an API key stored as an environment variable.

10.3 RAG Chain Construction

The full inference pipeline is assembled as a LangChain runnable chain. The chain takes the user's question as input, passes it through the retriever to obtain the relevant context, formats the context using the custom formatting function, assembles the prompt using the prompt template, and passes the assembled prompt to the language model. The output is the model's text response, extracted from the AIMessage object returned by the Groq API. This chain can be invoked with a single method call, abstracting the complexity of the multi-step pipeline behind a clean interface.

11. System Evaluation

Evaluating a RAG system requires assessing both the quality of the retrieval component and the quality of the generated responses. This project focuses on quantitative evaluation of the retrieval

component, using a set of 100 programmatically generated evaluation queries covering a range of common medical conditions.

11.1 Evaluation Dataset

The evaluation dataset consists of 100 queries constructed from a combination of ten medical topics and five question templates. The ten topics include eczema, anemia, migraine, asthma, rheumatoid arthritis, hypothyroidism, chronic kidney disease, osteoporosis, influenza, and stroke. The five question templates cover symptom inquiry, cause inquiry, diagnostic method inquiry, treatment inquiry, and complication inquiry. The queries are generated by applying each template to each topic in a cycling fashion, producing a diverse and representative set of evaluation queries that exercise different types of medical knowledge retrieval.

Each query is paired with an expected condition string that describes the key concepts that a correct response should reference. This expected condition is used to compute recall metrics by checking whether the retrieved document set contains text that includes the expected keywords.

11.2 Evaluation Metrics

The evaluation function computes three metrics across all 100 evaluation queries.

Hit Rate at K measures the proportion of queries for which the retriever returned at least one relevant document. A query is counted as a hit if any document is retrieved, which in this context means the retriever successfully found chunks related to the medical topic of the query. The Hit Rate reflects the system's ability to consistently engage with the medical domain of the question.

Recall at K measures the proportion of expected keywords that appear in the combined text of the retrieved documents. For each query, the expected condition string is split into individual words, and the fraction of those words that appear in the concatenated text of the top-k retrieved chunks is computed. The average of this fraction across all queries yields the Recall at K metric.

A higher Recall at K indicates that the retriever is consistently returning chunks that contain the specific medical concepts expected for each query.

Average Latency measures the mean time required to execute a retrieval query, reported in milliseconds. This metric is important for assessing the practical usability of the system in an interactive context. Retrieval latency that is too high would make the chatbot feel unresponsive, degrading the user experience.

12. User Interface

To make the medical chatbot accessible to non-technical users, a web-based graphical user interface was developed using the Gradio library. Gradio is a Python library designed specifically for building interactive machine learning demonstrations with minimal front-end development effort.

12.1 Gradio Interface

The Gradio interface wraps the `answer_question` function, which encapsulates the complete RAG pipeline from query reception to answer generation. The interface presents the user with a single text input field labeled `Enter Your Medical Question` and a corresponding text output field labeled `Answer`. When the user types a question and submits it, the interface invokes the `answer_question` function, which executes the full retrieval and generation pipeline and returns the model's response to be displayed in the output field.

The interface is launched using Gradio's built-in launch method, which starts a local web server and generates a shareable public URL via Gradio's tunneling service. This allows the interface to be accessed from any device with an internet connection, not only the machine running the notebook. In the Google Colab environment, this is particularly important, as Colab notebooks run on remote servers and the Gradio public URL provides the only means of browser-based interaction.

13. Technology Stack

The system is built on a set of well-established open-source libraries and external services, each chosen for a specific role in the pipeline. The following describes each technology in detail.

13.1 LangChain

LangChain is an open-source framework for building applications powered by language models. It provides a modular, composable set of abstractions for document loading, text splitting, embedding, vector storage, retrieval, prompt templating, and chain construction. In this project, LangChain serves as the primary integration layer, connecting the embedding model, the FAISS vector store, the prompt templates, and the language model into a coherent, maintainable pipeline. The use of LangChain's standardized interfaces means that individual components can be swapped or upgraded without requiring changes to the rest of the pipeline.

13.2 FAISS

FAISS (Facebook AI Similarity Search) is a library developed by Meta AI Research for efficient similarity search over dense vectors. It implements a variety of index structures that provide different trade-offs between search speed, memory usage, and recall accuracy. For this project, the exact flat index provided through LangChain's FAISS integration was used, performing exact nearest-neighbor search by scanning all stored vectors for each query. FAISS is capable of searching millions of vectors in milliseconds on modern hardware, making it well-suited to the scale of this project's corpus.

13.3 Sentence Transformers

Sentence Transformers is a Python library that provides pre-trained transformer models fine-tuned for generating semantically meaningful sentence embeddings. The library is built on top of the Hugging Face Transformers library and implements a Siamese network training procedure that trains models to produce embeddings that are close together for semantically similar sentences and far apart for semantically dissimilar sentences. The all-MiniLM-L6-v2

model used in this project was trained using this framework on a large dataset of sentence pairs and achieves strong performance on semantic similarity and information retrieval benchmarks.

13.4 PyPDF / LangChain PDF Loader

The medical textbook is loaded using the PyPDFLoader class from the LangChain community library, which internally uses the pypdf library to parse PDF files. PyPDF is a pure-Python library for reading and writing PDF documents. It extracts the text content of each page, along with metadata such as the page number and source file path. These metadata fields are critical for the citation functionality of the chatbot, as they enable the system to tell users which page of the medical book a given piece of information was retrieved from.

13.5 Groq and LLaMA 3.1

Groq is an AI inference company that operates a cloud API for serving large language models with exceptionally low latency, enabled by its custom Language Processing Unit hardware. The LLaMA 3.1 8B Instant model served through Groq is a Meta AI open-weight language model with eight billion parameters, belonging to the third generation of the LLaMA series. LLaMA 3.1 models are trained on multilingual text corpora and demonstrate strong performance on a wide variety of natural language understanding and generation tasks. The model is accessed via the LangChain Groq integration, which provides a ChatGroq class compatible with LangChain's chat model interface.

13.6 Gradio

Gradio is an open-source Python library for creating shareable web interfaces for machine learning models and data science applications. It enables developers to wrap any Python function in a browser-based UI with a small amount of code. Gradio supports a wide range of input and output types, including text, images, audio, and structured data. In this project, it is used to create a simple text-in, text-out interface that connects the user to the RAG pipeline without requiring any web development expertise or infrastructure setup.

13.7 Google Colab and Kaggle

The project uses two cloud-based notebook environments at different stages. The Kaggle notebook environment is used for the computationally intensive data preprocessing and vector database construction phase, as it provides free access to GPU and CPU compute resources with generous disk quotas. The FAISS index constructed in Kaggle is exported as a dataset and made available to the inference phase. The Google Colab environment is used for the inference and demonstration phase, where the pre-built FAISS index is loaded from Google Drive and used to power the RAG chain and the Gradio interface.

14. End-to-End Workflow Summary

The complete workflow of the project can be summarized in two major phases: an offline phase focused on knowledge base construction, and an online phase focused on query processing and answer generation.

14.1 Offline Phase: Knowledge Base Construction

The offline phase is executed once and produces the persistent vector database that underpins all subsequent inference. It begins with the acquisition of data from two sources: a cleaned clinical CSV dataset from Kaggle and a medical reference PDF book downloaded from Google Drive. The CSV data is loaded into a pandas DataFrame, unnecessary columns are removed, and each remaining row is serialized into a text string and wrapped in a LangChain Document object. The PDF is loaded page by page using PyPDFLoader, and document metadata is normalized to retain only the source path and page number.

Both document sets are then chunked using the RecursiveCharacterTextSplitter with a chunk size of 500 characters and an overlap of 50 characters. The resulting chunk sets are merged into a single corpus. Each chunk is embedded using the all-MiniLM-L6-v2 sentence transformer model to produce a 384-dimensional vector. A FAISS index is constructed from these vectors, with UUID-based deduplication applied to ensure index cleanliness. The completed index is saved to disk, compressed, and uploaded as a Kaggle dataset for use in the inference phase.

14.2 Online Phase: Query Processing and Answer Generation

The online phase is executed for every user query. It begins with loading the pre-built FAISS index from Google Drive into memory, together with the embedding model. When a user submits a question through the Gradio interface, the question is embedded into a 384-dimensional vector using the same sentence transformer model used during index construction. The FAISS retriever performs a similarity search and returns the five most semantically similar text chunks, each accompanied by its source and page number metadata.

These chunks are formatted into a structured context block with standardized citation prefixes. The context block and the user's question are inserted into the prompt template, which also contains the system instructions governing the model's behavior. The completed prompt is submitted to the LLaMA 3.1 8B Instant model via the Groq API. The model generates a response that synthesizes the retrieved information into a concise answer and includes citations to the source material. This response is returned to the Gradio interface and displayed to the user.

15. Challenges and Design Decisions

The development of this system required a number of non-trivial design decisions, each motivated by specific technical constraints or quality requirements encountered during the project.

15.1 Embedding Model Selection

The choice of embedding model required balancing semantic quality against computational cost. Larger embedding models, such as those based on BERT-large or RoBERTa, produce higher-dimensional vectors with potentially richer semantic representations but require significantly more memory and computation to run. The all-MiniLM-L6-v2 model was selected because it achieves strong benchmark performance while being small enough to run on CPU within the memory constraints of the Kaggle and Colab environments.

15.2 Chunk Size Optimization

Determining the appropriate chunk size required consideration of multiple factors. Smaller chunks are more specific and less likely to contain irrelevant information, but may lose important context that spans sentence boundaries. Larger chunks are more contextually complete but may contain so much information that the similarity score is diluted by irrelevant content within the chunk. The 500-character chunk size with 50-character overlap was selected as a practical compromise that worked well empirically across both the narrative text of the medical book and the structured text of the CSV serializations.

15.3 Data Source Heterogeneity

Combining data from a structured CSV dataset and a narrative PDF textbook introduced challenges related to the heterogeneity of the source formats. The CSV data, when serialized, produces short, dense, keyword-rich text strings that are very different in style from the longer, more discursive prose of the medical textbook. Despite this stylistic difference, both types of content are successfully embedded by the all-MiniLM-L6-v2 model into the same semantic space, enabling retrieval to draw on both sources for a given query. The unified indexing of heterogeneous sources is a key architectural strength of the system.

15.4 Hallucination Mitigation

Medical question answering carries a heightened responsibility for accuracy, as incorrect information could potentially influence health-related decisions. The RAG architecture inherently reduces hallucination risk by grounding the model's response in retrieved context. The system prompt further reinforces this by explicitly instructing the model to use only the provided context, to admit ignorance rather than speculate, and to cite sources. These design choices collectively minimize the risk that the model will generate plausible-sounding but medically incorrect information.

16. Limitations

Despite the system's capabilities, several important limitations should be acknowledged.

The system's knowledge is bounded by the content of the two source documents. Medical conditions, treatments, or findings not covered in the CSV dataset or the medical textbook cannot be answered from the knowledge base. The system will indicate that it does not know the answer in such cases, but this means that the breadth of coverage is inherently limited by the source selection.

The retrieval mechanism relies on semantic similarity between the query and the stored chunks. For highly technical or domain-specific queries that use terminology not well-represented in the embedding model's training data, retrieval quality may degrade. The all-MiniLM-L6-v2 model is a general-purpose sentence transformer and has not been fine-tuned specifically on medical text, which may limit its ability to capture the nuances of medical terminology.

The system is not designed to provide personalized medical advice and should not be used as a substitute for professional medical consultation. Its responses are based on general medical knowledge from the indexed sources and do not account for individual patient circumstances, medical history, or the dynamic nature of clinical guidelines.

The evaluation methodology, while quantitative, relies on keyword overlap as a proxy for correctness. A retrieved chunk may contain the expected keywords without fully answering the question, or a correct answer may use different terminology than the expected condition string. A more rigorous evaluation would involve human assessment of answer quality or automated evaluation using a reference LLM judge.

17. Conclusion

This project demonstrates the successful application of a Retrieval-Augmented Generation architecture to the domain of medical question answering. By combining a structured clinical dataset with a comprehensive medical textbook, encoding both sources into a semantically searchable FAISS vector database, and coupling the retrieval system with the LLaMA 3.1 language model served through Groq, the team has built a system that can respond to a wide range of medical queries with grounded, cited answers.

The project covers the complete lifecycle of an AI application, from data acquisition and preprocessing, through embedding and vector indexing, to retrieval, prompt engineering, language model inference, evaluation, and user interface deployment. Each stage was implemented with careful attention to the practical constraints of the development environment and the quality requirements of the medical domain.

The RAG approach provides several important advantages for this use case. It ensures that the system's responses are grounded in verified source material rather than solely in the parametric knowledge of the language model, reducing the risk of hallucination. It allows the knowledge base to be updated independently of the language model by rebuilding the vector index with new or revised sources. And it provides a natural mechanism for source citation, enabling users to trace each answer back to a specific passage in the source documents.

The modular architecture of the system also provides a strong foundation for future extension. The embedding model could be replaced with a medically fine-tuned alternative for improved retrieval quality. The knowledge base could be expanded with additional medical references. The language model could be upgraded as more capable models become available. And the evaluation framework could be enriched with human assessments and more sophisticated automatic metrics. Taken together, these possibilities make this project not only a complete working system but also a practical platform for continued development.